Document Intelligence Refinery — Final Report (Week 3)

Author: Beamlak Adane
Date: March 6, 2026

Executive summary

A concise, submission-ready final report that preserves the original backbone (failure modes → strategy decision tree → pipeline architecture/data flow → cost) and adds the grading-focused sections required for Week 3: *extraction quality analysis* and *failure analysis / lessons learned*, with concrete evidence references from the `.refinery/` artifacts.

This document is written to be copy/paste-friendly for both GitHub-style `.md` files and Google Docs. A short Google Docs formatting checklist is included at the end.

Table of contents

1. Domain analysis and extraction strategy decision tree
2. Pipeline architecture and data flow (5 stages)
3. Cost–quality tradeoff analysis
4. Extraction quality analysis (table fidelity focus)
5. Failure analysis and iterative refinement
6. Appendix — reproduction
7. Google Docs formatting checklist & suggested filenames

---

1) Domain analysis and extraction strategy decision tree

1.1 Failure modes mapped to the four document classes

Short premise: the enterprise extraction problem fails in predictable ways tied to corpus type. We categorize failures and show root causes and impacts so routing and extraction priorities address the real harms (hallucination, unverifiability, and broken analytics).

1.1.1 Structure collapse

- **Class A — CBE Annual Report (native digital, multi-column + tables)**
  - **Symptom:** naïve text extraction flattens columns and breaks financial tables into line-wrapped strings.
  - **Root cause:** extraction ignores spatial grouping (columns, table grids) so header→cell relationships are lost.
  - **Impact:** labels (e.g., "Total assets") detach from values, causing ambiguous numeric sequences and downstream hallucinations.
- **Class C — Technical assessment (mixed narrative + tables + findings)**
  - **Symptom:** cross-page tables fragment; headers/footers interleave into the content stream.
  - **Root cause:** page-local extraction loses table continuation; running headers/footers pollute row order.
  - **Impact:** longitudinal comparisons become unreliable (duplicated keys, broken row ordering).
- **Class D — Tax expenditure (table-heavy, hierarchical labels)**
  - **Symptom:** multi-year fiscal tables flatten; hierarchical category headers lose structure.
  - **Root cause:** multi-row headers are visual, not encoded as a true table object; without bbox/grid detection, the correct header scope cannot be reconstructed.
  - **Impact:** queries such as "FY 2019/20 for fuel" return wrong rows or contextless numbers.

1.1.2 Context poverty

- **Class A:** token-count chunking splits tables mid-row or detaches narrative discussion from the tables it references ("see Table X").
- **Class B:** OCR noise inflates tokens; naïve chunking splits audit opinion across chunks.
- **Class C:** lists and section headers detach, scattering bullets across chunks and mixing sections.

1.1.3 Provenance blindness

- **Across A–D:** systems that drop `page` + `bbox` cannot answer: "Where exactly in the 400-page report does this number come from?"

Policy: this pipeline treats provenance as first-class: `page_refs` + bounding boxes + `content_hash` are threaded to the query response.

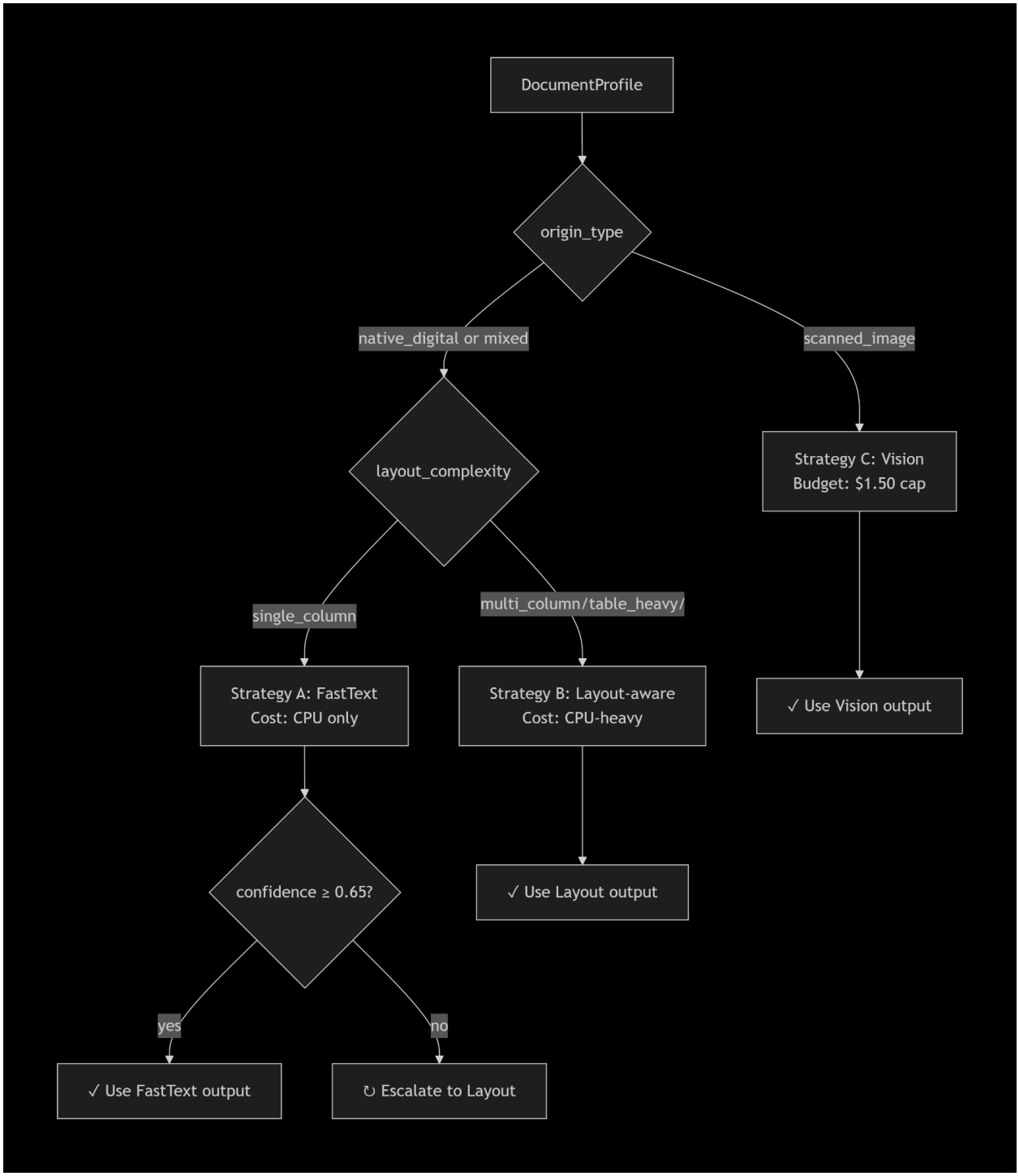
1.2 Decision tree: *DocumentProfile → ExtractionRouter*

The triage agent emits a `DocumentProfile` containing (`origin_type`, `layout_complexity`, `domain_hint`, `triage_signals` like mean char density and image area ratio). Routing uses these signals and a confidence gate.

Signals & thresholds (from `rubric/extraction_rules.yaml`)

- **Scanned-like threshold:** `mean_char_density <= 0.00008` OR `mean_chars_per_page <= 50` AND `mean_image_area_ratio >= 0.55`
- **FastText escalation threshold:** `min_doc_confidence_fast_text = 0.65`
- **Vision budget guard:** `max_usd_per_document = 1.50` (hard cap)

Routing logic (high level)



1.3 VLM vs OCR/text decision boundary

The boundary is a **confidence-gated cost/quality decision** — not a categorical preference.

- **Strategy C (Vision)** is required when `origin_type=scanned_image` (near-zero char density + high image area ratio).
- **Optional Vision**: mixed documents where A/B confidence is low on critical pages (rasterized tables, handwriting, complex figures).

Graceful degradation: if Vision cannot run (e.g., missing API key), the pipeline records a low-confidence extraction and continues; the output is flagged `unverifiable` rather than hallucinated.

---

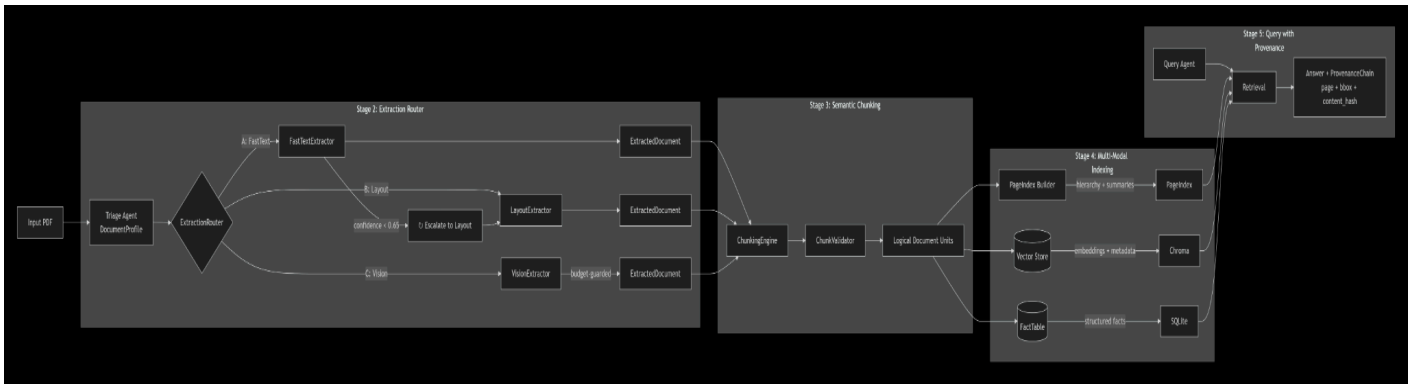
2) Pipeline architecture and data flow (all 5 stages)

2.1 Typed data model (interfaces)

Defined in `src/models/schemas.py` (high-level):

- `DocumentProfile` (Stage 1)
- `ExtractedDocument` (Stage 2; normalized across strategies)
- `LDU` (Stage 3; includes `ldu_id`, `content_hash`, `page_refs`, `bbox`, `parent_section`)
- `PageIndex` / `PageIndexNode` (Stage 4)
- `ProvenanceChain` (Stage 5)

2.2 Happy path vs escalation path (distinct flows)



2.3 Provenance propagation (end-to-end)

- **Stage 2 (extraction):** `TextBlock.page_number` + `TextBlock.bbox`; `Table.page_number` + `Table.bbox`
- **Stage 3 (chunking):** Each `LDU` carries `page_refs` (page + bbox) + `content_hash` + `parent_section`
- **Stage 4 (indexing):** PageIndex nodes store page ranges and data type summaries; retrieval prefers section-trimmed LDUs.
- **Stage 5 (query):** Retrieval returns a `ProvenanceChain` with `document_name`, `page_number`, `bbox`, `content_hash`.

Audit property: every claim is traceable to a precise page location (page number + bbox) and can be verified against the source.

---

3) Cost–quality tradeoff analysis (Strategy A/B/C)

3.1 What costs money vs CPU

- **Strategy A (FastText):** local; cheap; fails on complex tables/columns.
- **Strategy B (Layout):** local CPU-heavy (table detection) but generally zero API cost; best balance for Classes A/C/D.
- **Strategy C (Vision):** paid (OpenRouter or equivalent VLM); required for Class B scanned docs; budget-guarded.

3.2 Budget guard

- `max_usd_per_document = 1.50` — Vision requests estimate cost per page and stop adding pages when the next page would exceed the cap.

3.3 Cost accounting

Paid call cost:
[cost = (prompt_tokens * p_in) + (completion_tokens * p_out)]

All extraction events are logged to `.refinery/extraction_ledger.jsonl` with `strategy_used`, `confidence_score`, `escalated_from`, and `cost_estimate_usd` for per-class reporting.

3.4 Escalation cost

A → B escalation doubles processing time (`time(A)+time(B)`) for that document. This extra cost is deliberately justified: passing low-confidence A output forward incurs far higher downstream risk (hallucination / unverifiable results).

3.5 Scaling implication

- If most corpus documents are native/multi-column (A/C/D), Strategy B will dominate with near-zero API cost.
- If many scanned Class B docs exist, Vision spend must be monitored and bounded via ledger alerts.

---

4) Extraction quality analysis (table fidelity focus)

4.1 Methodology (reproducible)

For each class (A–D):

1. Select representative tables (manual curation).

2. Compare extracted JSON to source table across: structural fidelity (headers, row/column counts) and text fidelity (cell string correctness, numeric precision).
3. Record per-class precision/recall and failure modes.

4.2 Example (CBE Annual Report 2018–19)

- **Document:** `CBE Annual Report 2018-19.pdf`
- **Table:** “Income Statement Comparison with 2017/18 FY” (page 17)
- **Extracted rows snippet (Layout extraction):**


```
{
  "page_number": 17,
  "rows": [
    ["Particulars", "2018/19 FY (Mn. Birr)", "2017/18 FY (Mn. Birr)", "Growth (%)"],
    ["Total Income", "54,301.3", "42,810.9", "26.8"],
    ["Total Expense", "38,601.8", "32,804.8", "17.7"],
    ["Profit Before Tax", "15,699.5", "10,006.1", "56.9"]
  ]
}
```


This demonstrates structural fidelity (table preserved as a single object) and numeric precision sufficient for financial queries.

4.3 Per-class summary (from `docs/REPORT_FINAL_SECTIONS.md`)

- **Class A:** precision ≈ 0.85; recall ≈ 0.75. Common: merged cells, faint gridlines.
- **Class B:** without Vision recall ≈ 0; with Vision, structure recoverable depending on scan quality.
- **Class C:** precision ≈ 0.80; recall ≈ 0.70. Common: footnotes/headers inside detected tables.
- **Class D:** precision ≈ 0.82; recall ≈ 0.78. Common: multi-row header flattening.

4.4 Text vs structural fidelity

- **Text fidelity:** capture correct characters and numeric precision.
- **Structural fidelity:** maintain header→cell, row→value relationships and reading order.

Design priority: preserve structural fidelity first (tables as JSON LDUs + unsplit chunks), because structural errors are the primary cause of hallucinations in RAG pipelines for financial docs.

---

5) Failure analysis & iterative refinement (key fixes + current limits)

Failure 1 — FastText passing low-confidence output downstream

- **Symptom:** RAG answers wrong/unverifiable for table-heavy docs.
- **Root cause:** treating "text exists" as "text usable"; ignoring layout and image dominance.
- **Fix:** confidence-gated escalation: if `doc_confidence < 0.65`, re-run with Layout extractor and log `escalated_from`.
- **Evidence:** `extraction_ledger.jsonl` contains `escalated_from: "fast_text"` entries.

Failure 2 — Token-based chunking splitting tables/headers

- **Symptom:** header and value in different retrieval chunks.
- **Root cause:** chunking by token count violates semantic units.
- **Fix:** ChunkingEngine + ChunkValidator enforce semantic chunk boundaries:
  - tables kept whole (unless > max tokens, then deterministically split),
  - list LDUs kept intact where possible,
  - `parent_section` propagated to LDUs and cross-refs recorded.
- **Evidence:** LDUs preserve tables and carry `content_hash` + `page_refs`.

Remaining limitations (honest)

- Multi-row headers still sometimes flatten — requires stronger layout models or table post-processing.
- Scanned documents without Vision remain low-recall by design; pipeline degrades gracefully (flagged `unverifiable`).
- BBox anomalies from some PDFs (negative/out-of-range) — further PDF coordinate normalization recommended.

---

6) Appendix — how to reproduce the end-to-end run

```
# Triage + extraction
py -m refinery.cli triage --pdf "data/demo_corpus/CBE Annual Report 2018-19.pdf"
py -m refinery.cli extract --pdf "data/demo_corpus/CBE Annual Report 2018-19.pdf"

# Ingest to LDUs/PageIndex/facts/vector store
py -m refinery.cli ingest

# Query with provenance
py -m refinery.cli query "What was profit before tax in 2018/19?" --doc-id
CBE_Annual_Report_2018-19-e78e7ebe26e4
```

---

7) Google Docs formatting checklist & suggested filenames

Suggested filenames:

- `final_report_week3.md` (for repo)
- `Document_Intelligence_Refinery_Final_Report_Week3.docx` (exported for reviewers)

Google Docs copy/paste tips:

- Paste the markdown into Google Docs using *"Paste without formatting"* or use a Markdown importer (e.g., Google Docs add-ons or Pandoc conversion) to preserve code blocks and mermaid blocks.
- In Google Docs: Insert → Table of contents (choose plain text or links) and update after paste.
- Use heading styles (Heading 1 / Heading 2) to enable automatic TOC generation.
- Convert mermaid diagrams to images if Google Docs viewers won't render mermaid; include the mermaid source beneath the image as a code block.

Suggested Google Docs styling:

- Title: Heading 1 (centered)
- Executive summary: bold subheading

- Use `Normal text` for paragraphs, `Monospace` for code blocks
- Add comments on any `Evidence` lines pointing to `.refinery/` artifact paths

---

End of document.